



Défi TextMine 2026 : “ De la similarité sémantique aux grands modèles de langage ”

Maksim Koptelov

► To cite this version:

Maksim Koptelov. Défi TextMine 2026 : “ De la similarité sémantique aux grands modèles de langage ”. 26èmes Journées Francophones Extraction et Gestion des Connaissances, Jan 2026, Anglet, France. hal-05482654

HAL Id: hal-05482654

<https://hal.science/hal-05482654v1>

Submitted on 28 Jan 2026

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



HAL Authorization

Défi TextMine 2026 : « De la similarité sémantique aux grands modèles de langage »

Maksim Koptelov*

*UMR IATE, Univ. Montpellier, INRAE, Institut Agro, 34060, France
maksim.koptelov@inrae.fr

1 Introduction

Je suis actuellement chercheur postdoctoral à INRAE (l’Institut national de recherche pour l’agriculture, l’alimentation et l’environnement). Mes recherches portent sur diverses tâches en traitement automatique du langage naturel (TALN), en particulier la classification de textes, la reconnaissance d’entités nommées et des problématiques liées aux modèles de langues de grande taille (LLMs), telles que la génération augmentée par récupération (RAG). Par exemple, mes travaux incluent la construction de corpus thématiques (Koptelov et al., 2023), l’extraction d’informations à partir de documents structurés (Koptelov et al., 2025a) et la classification de textes à l’aide d’un modèle basé sur les transformers, fine-tuné sur des données synthétiquement générées avec des LLMs et RAG (Koptelov et al., 2025b).

Ma participation au défi TextMine 2026 (Lefeuvre et al., 2025) m’a permis d’appliquer mon expertise à un cas industriel réel. Je résume mes approches dans la section suivante et rends mon code¹ disponible publiquement.

2 Méthodes

Je propose d’évaluer 5 approches. La première est une méthode de base basée sur la similarité sémantique. Ensuite, la seconde est une approche utilisant un modèle de type BERT (Bidirectional Encoder Representations from Transformers) (Devlin et al., 2019). La troisième approche est basée sur un LLM accessible via l’API gratuite de l’Université de Montpellier (UM). Des méthodes hybrides, combinant ces différentes solutions, constituent ma quatrième approche. Enfin, j’évalue plusieurs LLMs propriétaires, dont ChatGPT-5 (OpenAI, 2025) et Grok-4 (xAI, 2025). En plus de la sélection du modèle le plus performant, je développe un pipeline RAG (Gao et al., 2023) visant à améliorer les performances des LLMs en intégrant une base de connaissances externe.

2.1 Méthode de base axée sur la similarité sémantique

Le défi consiste à identifier les bonnes extensions des acronymes à partir d’une liste d’options donnée. Chaque exemple d’entraînement est composé d’un tuple contenant un acronyme,

1. <https://github.com/koptelovmax/DefiTextMine26>

De la similarité sémantique aux grands modèles de langage

```
▼ "root" : { 3 items
  "text" :
    string "LRA limite de résistance des attelages PAR poste d'aiguillage et de régulation PL pleine
    ligne PN passage à niveau RFN réseau ferré national "
  "acronym" : string "PAR"
  ▼ "options" : { 4 items
    "Plan d'action régularité" : bool false
    "
    Poste d'aiguillage et de régulation : assure les commandes des installations de
    signalisation et la gestion de la circulation des huit lignes à grande vitesse
    "
    : bool true
    "PONT DE L'ARCHE" : bool false
    "Plan d'action régional" : bool false
  }
}
```

FIG. 1 – Exemple de tuple issu des données d'entraînement contenant le texte, l'acronyme et les formes étendues possibles avec annotation booléenne (Lefevre et al., 2025).

un segment de texte faisant référence à cet acronyme et une liste de formes étendues possibles annotées comme True ou False (Fig. 1). Les exemples de test ont la même structure, sauf que les options possibles ne sont pas annotées. La tâche consiste donc à déterminer, pour chaque exemple de test, quelles extensions candidates non annotées sont correctes. Au total, le jeu de données contient 492 exemples d'entraînement et 519 exemples de test.

Après avoir inspecté plusieurs exemples et effectué une analyse exploratoire initiale des données d'entraînement, j'ai observé les points suivants. Premièrement, le nombre d'options par exemple n'est pas constant et varie de 2 à 13. J'ai également constaté que certains exemples contiennent plus d'une réponse correcte. Cependant, le nombre maximal de réponses correctes est de deux, et cette situation n'apparaît que 9 fois sur les 492 exemples d'entraînement. En théorie, ce phénomène pourrait donc être ignoré si nécessaire. En revanche, 68 exemples n'ont aucune réponse correcte, ce qui signifie que ce cas ne doit pas être ignoré.

À partir de ces observations, j'ai conçu une méthode de base simple. L'idée est de calculer la similarité sémantique entre le texte de chaque exemple et chaque option candidate, puis de sélectionner l'option avec le score de similarité le plus élevé. Par *similarité sémantique*, j'entends la similarité cosinus entre les vecteurs d'embedding correspondants. Les vecteurs d'embedding pour le texte et les candidats sont obtenus à l'aide d'un modèle SentenceTransformer. Le modèle *dangvantuan/french-document-embedding* de HuggingFace (Wolf et al., 2020) a été sélectionné expérimentalement parmi ceux prenant en charge la langue française.

La similarité cosinus est calculée à l'aide de l'équation suivante :

$$\text{similarity}(A, B) = \frac{A \cdot B}{\|A\| \|B\|},$$

où A et B sont les vecteurs d'embedding correspondant au texte et à une option candidate.

Par exemple, les similarités sémantiques entre le texte et les extensions possibles de la Fig. 1 sont les suivantes :

— « Plan d'action régularité » : 0,66 ;

- « Poste d’aiguillage et de régulation : assure les commandes des installations de signalisation et la gestion de la circulation des huit lignes à grande vitesse » : 0,76 ;
- « PONT DE L’ARCHE » : 0,64 ;
- « Plan d’action régional » : 0,62.

La similarité la plus élevée correspond à la deuxième option, qui est la bonne réponse.

Cette méthode suppose que chaque segment possède exactement une réponse correcte. Comme elle ne comporte aucune phase d’apprentissage, je l’ai évaluée directement sur le jeu d’entraînement. L’approche a atteint un score F_1 de 0,58, ce qui constitue un point de départ raisonnable. Ce résultat relativement faible s’explique par la principale limitation de la méthode : elle ne tient pas compte du fait que le nombre de réponses correctes peut varier d’un exemple à l’autre.

2.2 Approche basée sur les transformers

La deuxième approche que j’ai explorée consiste à modéliser la tâche comme un problème de classification de textes en utilisant une architecture basée sur les transformers de la famille BERT, plus précisément CamemBERT (Martin et al., 2019), puisque les données sont en français. Les modèles basés sur les transformers ont montré de bonnes performances en classification de textes dans mes travaux précédents (Koptelov et al., 2023, 2025b,a), notamment pour des données en français (Koptelov et al., 2023, 2025a). Cependant, dans ce type d’approche, la segmentation des données constitue une étape de conception cruciale.

Suivant la méthodologie de (Koptelov et al., 2023), je représente chaque exemple en concaténant le texte du segment, l’acronyme et une option candidate. Avant de construire ces entrées, j’ai divisé le jeu de données d’entraînement original en ensembles d’entraînement, de validation et de test avec des proportions de 64%, 16% et 20%, respectivement. Cela a donné 1434 exemples d’entraînement, 325 exemples de validation et 418 exemples de test. L’ensemble d’entraînement a été utilisé pour fine-tuner un classificateur binaire basé sur CamemBERT, l’ensemble de validation a servi à l’ajustement des hyperparamètres, et l’ensemble de test a été conservé pour l’évaluation sur des données non vues.

J’ai fine-tuné le modèle en utilisant l’optimiseur AdamW avec un taux d’apprentissage de $2 \cdot 10^{-5}$, $\epsilon = 10^{-8}$ et 10 epochs. La meilleure exécution a atteint un score F_1 de 0,89 sur l’ensemble de validation à la huitième époque. L’évaluation sur l’ensemble de test a donné un F_1 de 0,81. Bien que ce résultat soit nettement supérieur à celui de la méthode de base, il n’est pas directement comparable, car la baseline a été évaluée sur une partition différente. Néanmoins, cette méthode est plus avancée que la baseline, car elle repose sur un vocabulaire représentant chaque classe et peut identifier à la fois les cas sans solution et les cas à solutions multiples.

2.3 Approche basée sur un LLM open-source

Les LLMs ont montré des résultats très prometteurs en classification de textes (Zhao et al., 2023) et en désambiguïsation d’acronymes (Hosseini et al., 2024). Dans le but de développer une autre baseline basée sur un LLM, j’ai expérimenté le recours à un LLM open-source.

Le principal inconvénient des LLMs propriétaires est le coût élevé de l’utilisation de leur API, c’est pourquoi j’ai choisi d’utiliser le modèle Codestral-22B, disponible gratuitement

à l'UM². Même si ce modèle ne semble pas particulièrement adapté à cette tâche, je l'ai testé comme baseline supplémentaire et comme opportunité de développer des stratégies de prompting spécifiques à ce défi.

Les aspects les plus difficiles de cette approche ont été l'ingénierie du prompt et la conception d'un parser de résultats fiable. Pour le prompt, j'ai finalement utilisé la requête suivante : *"Find what is "{ACRONYM}" in this phrase : "{TEXT}". Evaluate the following options by giving the boolean list as an answer : "{OPTIONS}". Output only the boolean list. Do not output any explanation nor other text"*. J'ai utilisé l'anglais pour le prompt, car les LLMs répondent généralement plus précisément aux prompts en anglais, ce qui a été confirmé par des expériences ultérieures. Pour l'implémentation du parser, j'ai tiré parti des capacités de sortie structurée (« vibe coding ») de ChatGPT-5.

L'évaluation de cette approche sur les données d'entraînement a donné un score F_1 de 0,48, inférieur à la baseline. Ce résultat relativement faible peut s'expliquer par le fait que le modèle Codestral n'est pas particulièrement adapté à ce type de tâche, ayant été principalement entraîné pour des benchmarks de complétion de code. Par conséquent, il manque de connaissances spécifiques au domaine ferroviaire, notamment dans le contexte français. Néanmoins, il constitue une alternative supplémentaire à la méthode de similarité sémantique et peut servir de baseline complémentaire.

2.4 Approche hybride

L'évaluation de la méthode basée sur les transformers sur le jeu de test public de Kaggle a donné un score F_1 de 0,51, tandis que la méthode de base a obtenu $F_1 = 0,55$ et l'approche LLM basée sur Codestral a atteint $F_1 = 0,47$. Fait surprenant, la méthode de base utilisant la similarité sémantique a montré la meilleure performance malgré ses limitations : son incapacité à détecter les exemples sans réponse correcte ou avec plusieurs réponses correctes.

Pour pallier ces limites, j'ai développé une approche *hybride*. Dans cette méthode, le classificateur basé sur les transformers est utilisé pour détecter les cas sans solution valide, le LLM Codestral sert à identifier les cas avec plusieurs réponses correctes, et la méthode de similarité sémantique est appliquée pour déterminer la meilleure réponse unique (ou deux réponses au maximum dans les cas à solutions multiples). Pour mettre en œuvre cette approche, j'ai utilisé comme entrées les données de test annotées par la méthode basée sur les transformers et celles annotées par le LLM Codestral. Pour cette raison, aucune séparation train/test n'a été nécessaire et l'évaluation a été réalisée directement sur le jeu de test public. Le calcul de la similarité sémantique a suivi la procédure décrite dans la Section 2.1.

Cette stratégie hybride a atteint un score F_1 de 0,58 sur le leaderboard public de Kaggle. Il s'agit de l'approche la plus performante jusqu'à présent, répondant aux limitations de toutes les méthodes précédentes.

2.5 LLMs propriétaires et RAG

Des expérimentations supplémentaires avec l'approche hybride (par exemple, en utilisant la méthode basée sur les transformers pour prédire les solutions multiples et la méthode Codestral pour détecter les cas sans solution) n'ont pas conduit à une amélioration des performances.

2. <https://isdms-chat.crocc.meso.umontpellier.fr/>

TAB. 1 – Résultats d'évaluation du modèle open-source (Codestral-22B) et des LLMs propriétaires sur le jeu de test, ainsi que les coûts associés à l'usage de l'API.

Modèle	Évaluation sur le jeu de test		Coût API, USD
	Score public	Score privé	
Codestral-22B	0,47016	0,51263	–
GPT-5 nano	0,58695	0,62056	2,49
GPT-5 mini	0,74377	0,73779	2,72
GPT-5	0,80068	0,81109	4,31
GPT-5 + RAG	0,81292	0,78125	3,68
Grok-4 fast	0,72759	0,75310	2,14
Grok-4 fast + RAG	0,71381	0,68656	0,24
Grok-4 + RAG	0,67957	0,65831	21,39

J'ai donc évalué des LLMs propriétaires : ChatGPT-5 et Grok-4, récemment publiés et ayant montré des performances exceptionnelles sur des benchmarks publics (Sahil, 2025).

En utilisant le même pipeline et le même prompt que pour le modèle Codestral, j'ai évalué plusieurs modèles récents d'OpenAI et de xAI : GPT-5 nano (*gpt-5-nano-2025-08-07*), GPT-5 mini (*gpt-5-mini-2025-08-07*), GPT-5 (*gpt-5-2025-08-07*), Grok-4 fast (*grok-4-fast*) et Grok-4 (*grok-4-0709*). Les performances obtenues sont résumées dans le Tableau 1. Pour comparaison en termes de rapport coût-efficacité, le tableau présente également les coûts associés à l'utilisation de l'API.

Comme le montre le tableau, Grok-4 fast ($F_1 = 0,73$ et $0,75$ sur les jeux de test public et privé, respectivement) surpasse GPT-5 nano ($F_1 = 0,59$ et $0,62$) et est plus économique que GPT-5 nano et GPT-5 mini (0,24 USD pour Grok-4 fast contre 2,49 et 2,72 USD pour GPT-5 nano et mini). Cependant, parmi les modèles full-scale, GPT-5 offre à la fois de meilleures performances ($F_1 = 0,80$ et $0,81$) et des coûts d'API inférieurs (4,31 USD) par rapport à Grok-4, dont l'évaluation (sans RAG) n'a pas pu être complétée en raison de coûts d'API extrêmement élevés.

Pour améliorer davantage les résultats, j'ai expérimenté l'intégration d'un pipeline RAG (Gao et al., 2023). Cette technique enrichit le prompt avec un contexte sémantiquement pertinent provenant d'une source de connaissances externe. Comme source, j'ai utilisé une base de données publique d'acronymes de la SNCF³, qui contient 9 502 acronymes avec leurs expansions dans différents formats (CSV, JSON, etc.). J'ai utilisé la version CSV et effectué une recherche de similarité pour récupérer les 3 expansions les plus pertinentes⁴ pour chaque acronyme, qui ont ensuite été injectées dans le prompt enrichi par RAG. La recherche dans la base de données a été effectuée en comparant la similarité sémantique entre le texte du prompt et chaque entrée de la base (contenant un acronyme et son expansion). La méthode de recherche par similarité était identique à celle utilisée dans l'approche de base décrite à la Section 2.1.

Un défi majeur dans cette approche a été le choix d'un modèle d'embedding approprié. L'évaluation expérimentale a conduit au choix du modèle intfloat/multilingual-e5-base de HuggingFace (Wolf et al., 2020), qui supporte la langue française.

3. <https://ressources.data.sncf.com/explore/dataset/lexique-des-acronymes-sncf/>

4. déterminées expérimentalement en comparant les 2, 3 et 4 documents les plus pertinents

L'intégration de RAG avec le modèle GPT-5 full-scale a permis d'améliorer ses performances à $F_1 = 0,81292$ et de réduire les coûts d'API de 4,31 à 3,68 USD sur le jeu de test public. La meilleure performance s'explique par un pipeline plus avancé : bien que le LLM ait été entraîné sur un large instantané des ressources d'Internet, la version API ne permet pas la recherche web sans recourir à des moteurs commerciaux payants. RAG compense cette limitation en injectant directement dans le prompt les acronymes les plus pertinents et leurs expansions. La diminution des coûts d'API résulte de l'utilisation d'un pipeline RAG plus efficace.

3 Conclusion

Dans ce travail, j'ai présenté un aperçu de plusieurs approches pour relever le défi TextMine 2026, allant des méthodes de similarité sémantique aux modèles de langage de grande taille propriétaires combinés avec RAG. Le modèle le plus performant sur la partition publique est GPT-5 avec RAG, atteignant un score F_1 de 0,81292. Sur la partition privée, le meilleur résultat a été obtenu avec GPT-5 sans RAG, avec un F_1 de 0,81109.

Le GPT-5 full-scale s'est révélé être le modèle le plus avancé, démontrant les capacités de raisonnement les plus solides et contenant les connaissances les plus pertinentes pour la tâche comparé aux autres LLMs testés. Une autre observation est que le pipeline RAG n'améliore pas systématiquement les performances, mais qu'il réduit la charge de raisonnement du modèle. En conséquence, le modèle génère des réponses plus courtes et moins coûteuses, même lorsque le prompt est plus long, ce qui conduit à une diminution des coûts d'API.

La principale conclusion de cette étude est que le défi TextMine constitue un excellent benchmark pour l'évaluation des LLMs modernes. Il s'avère suffisamment exigeant même pour des modèles full-scale haut de gamme et de pointe.

Références

- Devlin, J., M.-W. Chang, K. Lee, et K. Toutanova (2019). Bert : Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics : human language technologies, volume 1 (long and short papers)*, pp. 4171–4186.
- Gao, Y., Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, et H. Wang (2023). Retrieval-augmented generation for large language models : A survey. *arXiv preprint arXiv :2312.10997*.
- Hosseini, M., M. Hosseini, et R. Javidan (2024). Leveraging large language models for clinical abbreviation disambiguation. *Journal of medical systems* 48(1), 27.
- Koptelov, M., M. Holveck, B. Cremilleux, J. Reynaud, M. Roche, et M. Teisseire (2023). A manually annotated corpus in french for the study of urbanization and the natural risk prevention. *Scientific Data* 10(1), 818.
- Koptelov, M., M. Holveck, B. Cremilleux, J. Reynaud, M. Roche, et M. Teisseire (2025a). (semi-) automatic extraction of urban planning rules in french for better management of land artificialization. *SN Computer Science* 6(2), 115.

- Koptelov, M., J. Linck, P. Bisquert, P. Buche, et M. Roche (2025b). Agricode : Automated coding for qualitative research and its application to the valorization of agricultural residues. *SoftwareX* 31, 102258.
- Lefeuvre, L., C. Reutenauer, A. Guille, P. Cuxac, et C. Lopez (2025). Défi TextMine / EGC 2026. <https://kaggle.com/competitions/defi-text-mine-egc-2026>. Kaggle.
- Martin, L., B. Muller, P. J. O. Suárez, Y. Dupont, L. Romary, É. V. de La Clergerie, D. Seddah, et B. Sagot (2019). Camembert : a tasty french language model. *arXiv preprint arXiv :1911.03894*.
- OpenAI (2025). Introducing gpt-5. <https://openai.com/index/introducing-gpt-5/>.
- Sahil, N. (2025). LLM Benchmarking : Compare Top AI Models for Your Specific Needs. <https://futureagi.com/blogs/llm-benchmarking-compare-2025>.
- Wolf, T., L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, et al. (2020). Transformers : State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing : System Demonstrations*, pp. 38–45. Association for Computational Linguistics.
- xAI (2025). Grok 4. <https://x.ai/news/grok-4>.
- Zhao, B., W. Jin, J. Del Ser, et G. Yang (2023). Chatagri : Exploring potentials of chatgpt on cross-linguistic agricultural text classification. *Neurocomputing* 557, 126708.

Summary

This work explores different methods for the TextMine 2026 challenge, from simple semantic similarity to transformer-based models and Large Language Models (LLMs). I evaluate open-source and proprietary LLMs, including hybrid and RAG-enhanced approaches, and compare their performance on both public and private test splits. The best performance is achieved by GPT-5 combined with a RAG pipeline, demonstrating that TextMine is challenging enough to be an excellent benchmark for modern LLMs.